

Solution Architecture

This document describes the technical architecture of OptiCrop, covering the component structure, module design, data pipeline, and deployment topology.

Technical Architecture

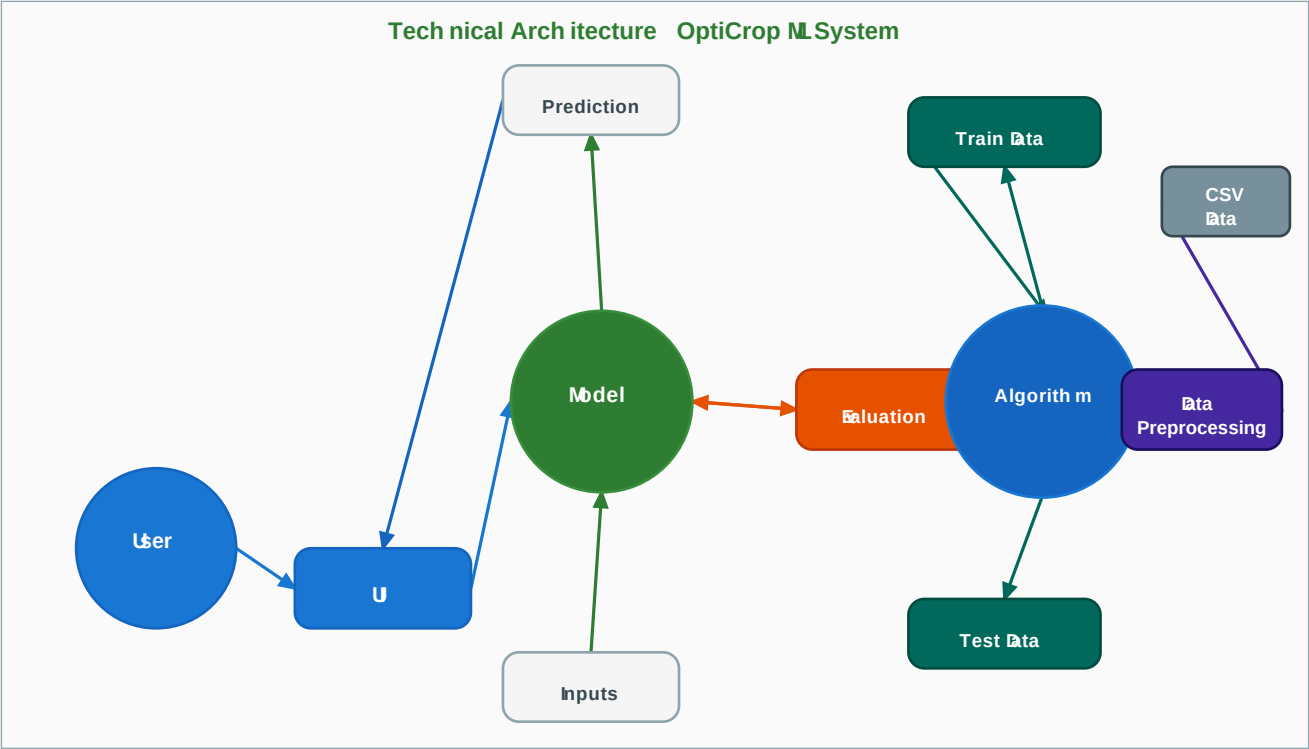


Figure 3: Technical Architecture OptiCrop ML System

Three-Tier System Architecture

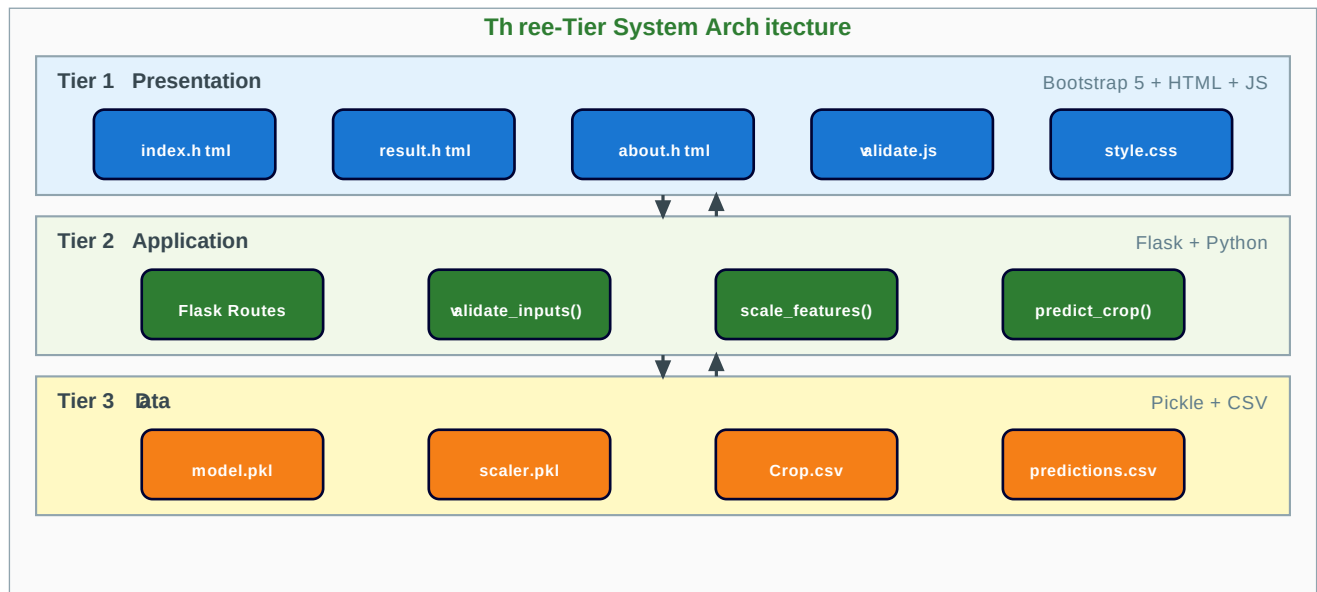


Figure 4: Three-Tier Architecture (Presentation Application Data)

Module Structure

Path	Description
opticrop/	Root project directory
app.py	Flask application factory routes, error handlers
requirements.txt	All Python dependencies with pinned versions
models/model.pkl	Trained Random Forest classifier (scikit-learn)
models/scaler.pkl	Fitted StandardScaler for input normalisation
data/Crop_recommendation.csv	Raw dataset: 2,200 rows ×8 columns
notebooks/01_Model_Training.ipynb	Model comparison, training, evaluation, saving
app/templates/index.html	Home page input form for 7 parameters
app/templates/result.html	Result page crop recommendation display
app/static/css/style.css	Custom CSS overriding Bootstrap defaults
app/ml/predictor.py	<code>load_model()</code> , <code>predict_crop()</code> , <code>get_confidence()</code>
app/ml/preprocess.py	<code>validate_inputs()</code> , <code>scale_features()</code>
logs/predictions.csv	Timestamped log of all prediction requests

Request-Response Flow

Step 1: Browser sends HTTP GET / Flask renders index.html with input form

Step 2: User fills form and clicks Predict Browser validates (JS) HTTP POST /predict

Step 3: Flask route calls `validate_inputs(request.form)` raises `ValidationError` if invalid

Step 4: `scale_features(validated_dict)` loads `scaler.pkl` returns numpy array (1,7)

Step 5: `predict_crop(scaled_arr)` loads `model.pkl` returns (crop_label, confidence)

Step 6: Flask logs request to `logs/predictions.csv`

Step 7: Flask renders `result.html` with crop_label, confidence, and crop metadata

Step 8: Browser displays recommendation page to user